Multithreading vs Multiprocessing

PERFORMANCE STUDY

CS 301 · Parallel Computing · Presented by Alex Johnson · Spring 2025

Why Parallelism Matters

Concurrency vs Parallelism

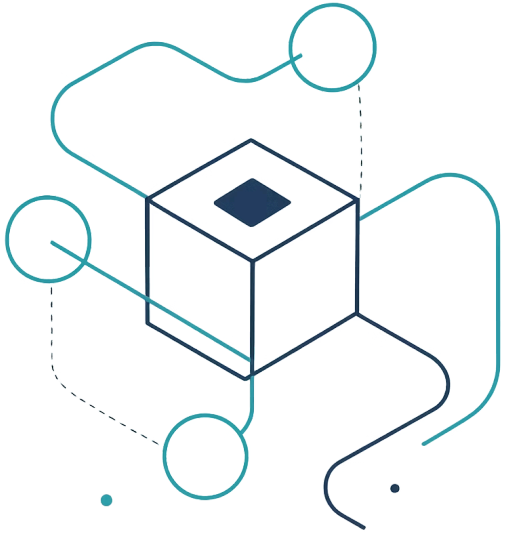
Concurrency is about dealing with multiple tasks at once — interleaving execution to make progress on several things.

Parallelism is about doing multiple things *simultaneously* — true simultaneous execution on multiple CPU cores.

Why It Matters Today

- Modern CPUs ship with 8–32+ cores
- Single-threaded performance has plateaued
- Servers handle thousands of concurrent requests
- Data-intensive workloads demand throughput

What Is Multithreading?



Definition

Multiple **threads** execute concurrently within a **single process**. Threads share the same memory space, enabling fast data exchange.

Shared Memory

Heap, globals, and file descriptors shared across all threads

Lightweight

Low creation overhead; context switches are fast

GIL Constraint

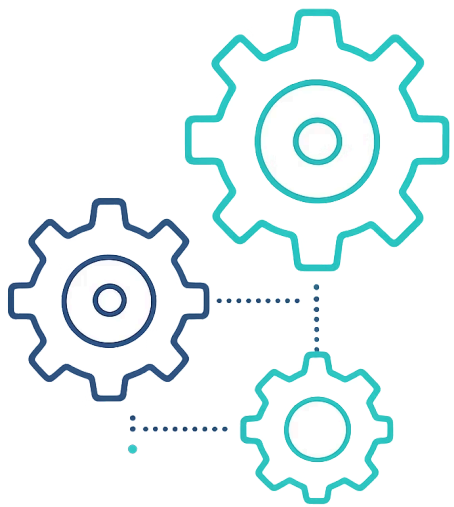
Python's Global Interpreter Lock limits true CPU parallelism

What Is Multiprocessing?

Definition

Multiple **independent processes** run simultaneously, each with its own memory space and Python interpreter instance.

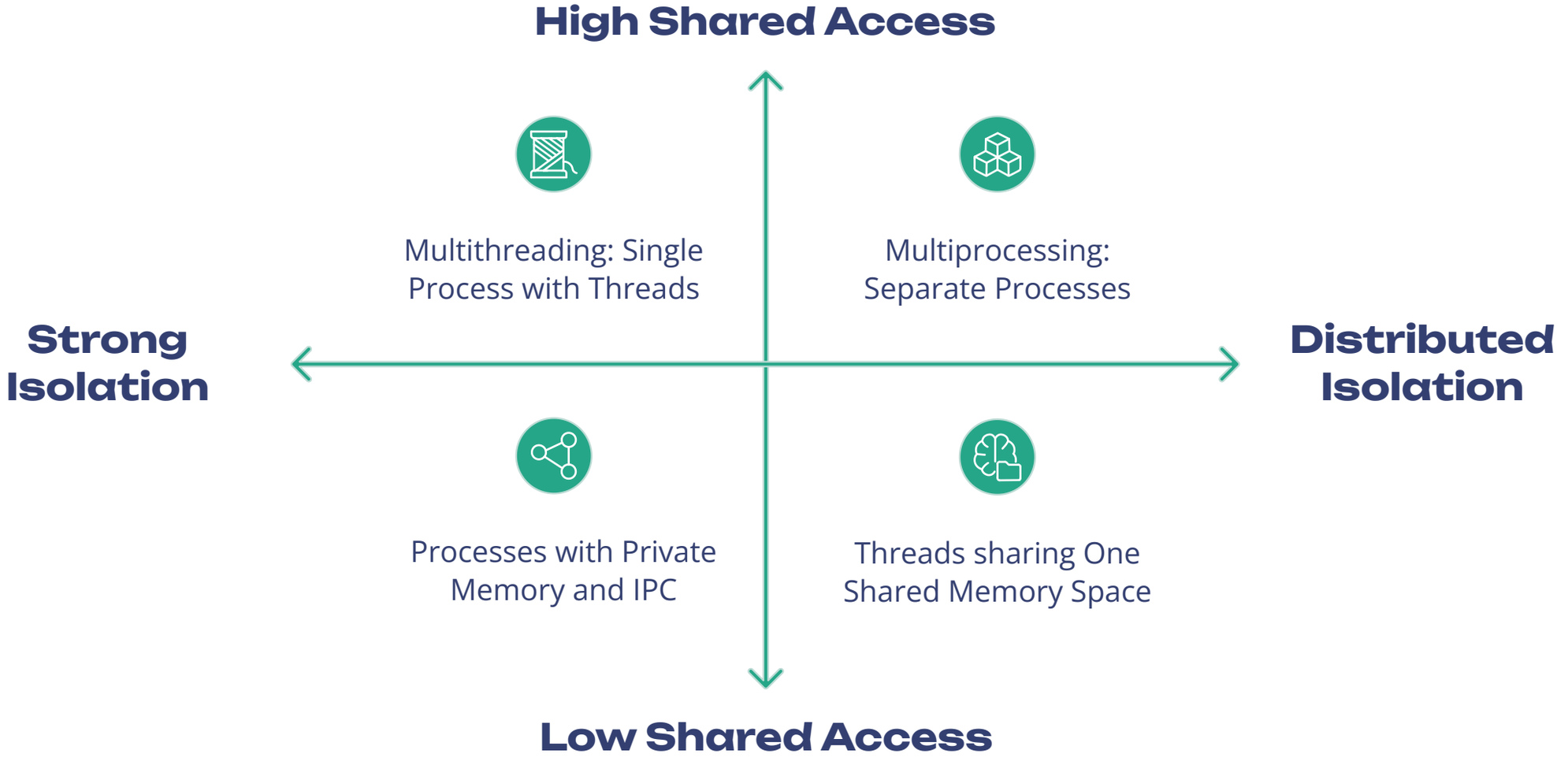
- Bypasses the GIL entirely — true CPU parallelism
- Processes do not share memory by default
- Communication via pipes, queues, or shared memory
- Higher creation overhead than threads



Key Differences at a Glance

Dimension	Multithreading	Multiprocessing
Memory	Shared address space	Separate address spaces
Creation Overhead	Low	High
Communication	Direct (shared vars)	IPC: pipes / queues
GIL Impact (Python)	Blocked for CPU tasks	Not affected
Best For	I/O-bound tasks	CPU-bound tasks
Fault Isolation	Low — shared crash	High — isolated crash

Architecture Diagram



Threads share a single memory space — enabling fast but risky data sharing. Processes maintain strict isolation, requiring explicit inter-process communication (IPC) but providing stronger fault boundaries.

Performance Factors

CPU-Bound Tasks

Heavy computation (e.g., matrix ops, compression). Multiprocessing wins — bypasses GIL and fully utilises multiple cores.

I/O-Bound Tasks

Waiting on disk, network, or APIs. Multithreading wins — threads yield during waits, allowing others to proceed.

Context Switching

Thread switches are cheaper than process switches. Excessive context switching adds latency and degrades performance.

Spawn Overhead

Processes require duplicating memory (fork/spawn). Threads are faster to create but must synchronise shared state.

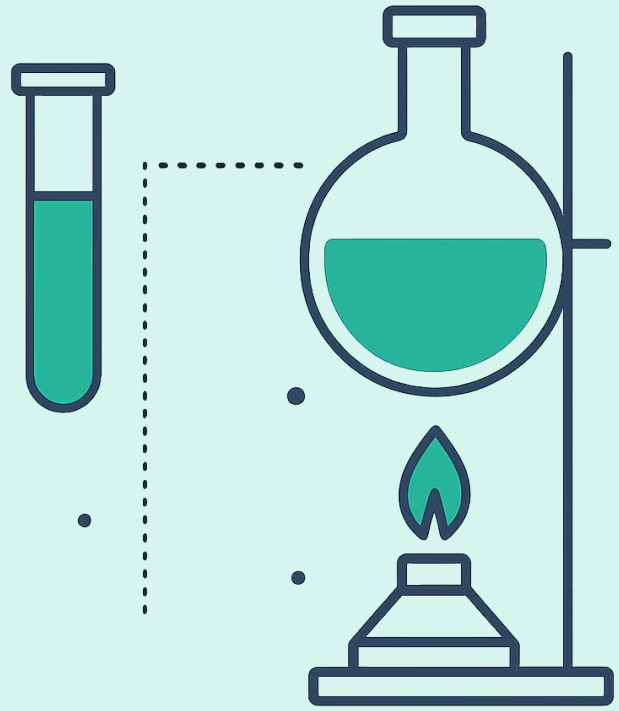
Experimental Setup

Environment

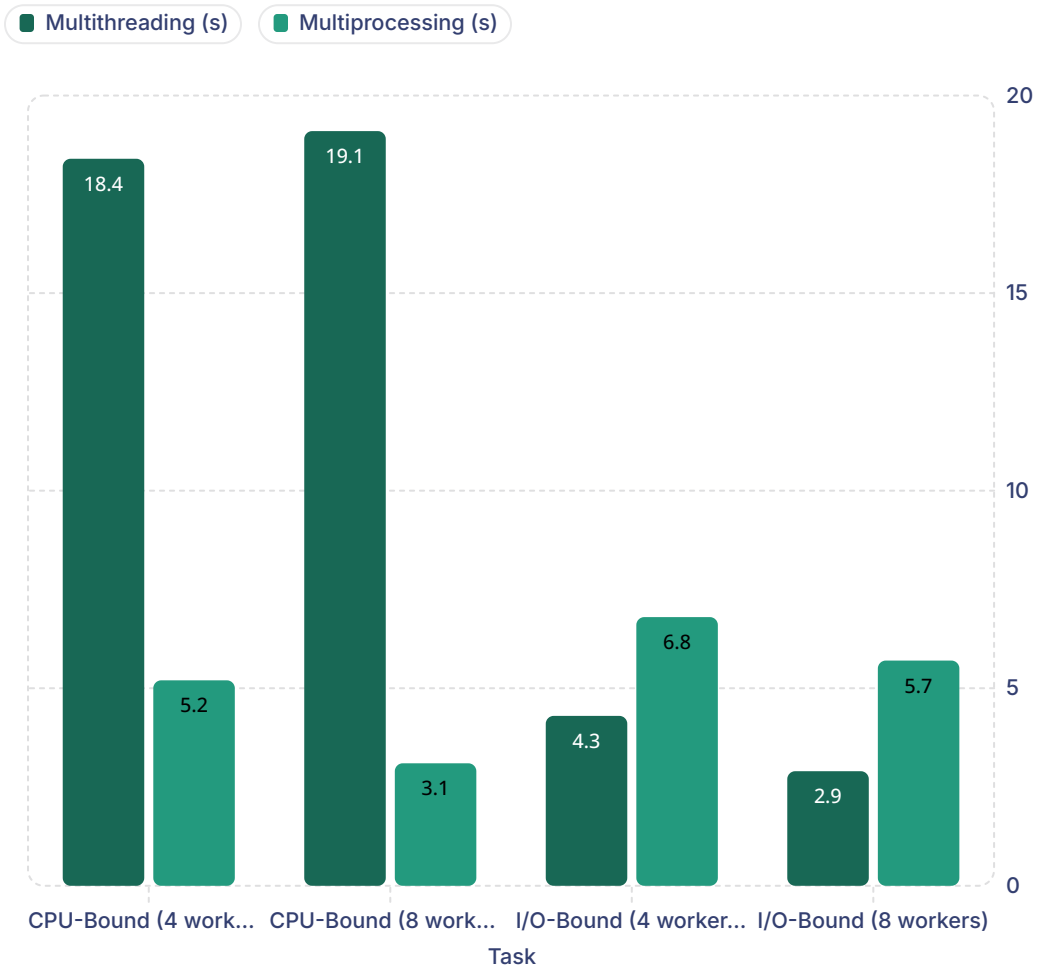
- **Language:** Python 3.11
- **Libraries:** `threading`, `multiprocessing`
- **Hardware:** 8-core Intel i7, 16 GB RAM
- **OS:** Ubuntu 22.04 LTS

Tasks & Metrics

- **CPU-bound:** Prime number sieve ($N = 10^7$)
- **I/O-bound:** Reading 500 files × 1 MB each
- **Workers tested:** 1, 2, 4, 8 threads/processes
- **Metrics:** Wall-clock time, CPU utilisation



Results & Analysis



Key Observations

- Multiprocessing is **~6× faster** on CPU-bound tasks with 8 workers
- Multithreading delivers **~40% speedup** on I/O-bound tasks at 8 workers
- Threading shows *minimal gain* on CPU tasks — GIL is the bottleneck
- Process spawn overhead is visible at lower worker counts

Execution time in seconds. Lower is better.

Conclusion

Use Multithreading When...

- Tasks are I/O-bound (network, disk)
- Low memory overhead is a priority
- Shared state is manageable

Use Multiprocessing When...

- Tasks are CPU-bound (heavy computation)
- True parallelism is required
- Fault isolation matters

Summary: Neither approach is universally superior — the optimal choice depends on your task profile. Profile first, then parallelise.